... and html-attr-escape.html.twig:

```
{# views/html-attr-escape.html.twig #}

<img src="{{ srcAttr|e('html_attr') }}"/>
$ curl http://localhost:8080/index.php
body\20 \3E \20 main\20 \2E link {
    font-weight: bold;
}

<img src="x&amp;quot;&amp;#x20;onerror&amp;quot;&amp;#x3D;alert&amp;#x28;&amp;#x27;script&amp;#x20;
injection&amp;#x27;&amp;#x29;&amp;quot;"/>
```

The full list of Twig escape contexts are:

Table 11-27. *Twig escape contexts*

`html`	Entity encoding
`js`	Backslash escaping, making data safe for use in JS or JSON output
`css`	CSS hexadecimal escape syntax
`url`	URL encoding, e.g., for use with query parameters
`html_attr`	Treats string as a single attribute value

You can also control escaping at a template block level by using the `{% autoescape %}` `{% endautoescape %}` tags; all content between these tags will be subjected to autoescaping regardless of any global setting. You can use these tags to switch strategies: for example, `{% autoescape 'js' %}` will apply the `js` escape context, `{% autoescape false %}` will disable autoescaping, and so on. Twig is also intelligent enough to only apply escaping once; an `{% autoescape %}` block inside a template already rendering with escaping enabled will not trigger a second pass (which would result in "double escaping" and mangled output).

Twig Filtering

Similar to escaping data, filters can be applied in Twig data expressions in a similar way using the `|` operator followed by the filter name (`|escape` is just a very specialized filter). Filters allow a degree of data processing in order to do things like transform data to strings or apply regional formatting for dates and numbers. For example, `{{ "Hello\`